Report on

“Tutorials”

For

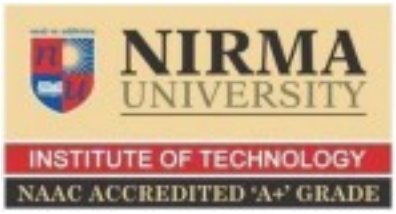
Software Project Management - 4CS504ME25 B.

Tech. Semester IV

Prepared By :

Gandhi Urva Yogeshkumar (23BCE078)

Gajnotar Rakshit (23BCE077)



Institute of Technology,
Nirma University
Ahmedabad - 382481

October 2026

Aim

To present a complete Software Project Management portfolio and capstone project report for the Hospital Management System (HMS), integrating planning, estimation, scheduling, risk management, quality assurance, SCM and project closure documentation.

Objective

Consolidate the complete software project managment lifecycle artifacts from Tutorials T1 through T14 into an integrated capstone case study.

Synthesize project scope, architectural baselines, software sizing metrics (LOC, FP, UCP) and quantitative estimation models (COCOMO II and Make-vs-Buy analysis).

Evaluate schedule performance, critical path tracking, earned value analysis and the fast-track schedule recovery executed between Weeks 12 and 14.

Review project risk governance, risk burndown trajectories and healthcare operational security controls.

Document quality engineering outcomes, testing rigor (22 test suites, 340 tests), Defect Removal Efficiency (91.43%) and software project audit compliance.

Formalize project closure documentation including deliverable verification matrices, schedule and effort variance analysis, post-mortem lessons learned and customer handover acceptance.

1. Executive Summary and Project Charter

PulseCare Hospital Management System (HMS) is a modular, web-based hospital administration platform developed to digitize outpatient clinical workflows, streamline patient record administration, automate doctor consultation scheduling and ensure accurate, itemized financial billing. Traditional hospital workflows often suffer from fragmented paper records, lost patient histories, un-tracked physician cancellations and manual billing calculation errors that impede patient care.

The project was executed over a 16-week academic lifecycle by a two-student engineering team: Gandhi Urva Yogeshkumar (23BCE078) serving as Backend and Architecture Lead and Gajnotar Rakshit (23BCE077) serving as Frontend and UI/UX Lead. The system was implemented as a production-grade TypeScript application powered by Express 5, Mongoose 8, MongoDB, Redis, React 18, Vite, TanStack React Query v5 and TailwindCSS. The platform achieves strict Role-Based Access Control (RBAC) across five user roles: Administrator, Doctor, Nurse, Patient and Receptionist.

1.1 High-Level Project Scorecard

Project Dimension	Baseline Target / Metric	Final Achieved Outcome	Status Assessment
Project Lifecycle Duration	16 Calendar Weeks (8 Sprints)	16.0 Calendar Weeks	Delivered on Schedule
Total Expended Effort	8.00 Person-Months baseline	8.86 Person-Months (incl. reserve)	Within Approved Budget (+10.7%)
Total Codebase Volume	25,000 Lines of Code	28,500 Lines of Code (TS/React)	Scope Complete (+14.0%)
Core Functional Modules	4 Core Clinical Modules	4 Core Modules + Auth & Reporting	100% Deliverables Verified
Automated Test Suites	>= 85.0% Code Coverage	90.2% Coverage (340 / 340 Passed)	Exceeded Quality Goal
Defect Removal Efficiency	>= 90.0% DRE	91.43% DRE (32 of 35 defects caught)	Exceeded Target

Project Dimension	Baseline Target / Metric	Final Achieved Outcome	Status Assessment
Software Project Audit	>= 95.0% Compliance	96.43% Post-CAPA Compliance	Audited and Approved (M5)

2. Project Planning, Work Breakdown and Sizing Synthesis

Project initiation (T1, T2 and T3) selected a two-week iterative Agile Scrum development process model embedded within a 16-week academic framework. Agile iterations accommodated requirements refinement from clinical mentors while milestone quality gates (M1 through M6) protected our semester submission deadline.

2.1 Work Breakdown Structure (WBS) Synthesis

As established in T4, project scope was decomposed into nine major deliverables and 23 discrete work packages:

- Deliverable D1 (Requirements & Feasibility): Requirements engineering, stakeholder interviews, IEEE 830 SRS authoring and acceptance criteria definition.
- Deliverable D2 (System Architecture): Modular TypeScript service architechture, MongoDB schemas, Redis caching and Docker container setup.
- Deliverable D3 (Patient Module): Patient demographic registration, medical history records, search indexing and ID card generation.
- Deliverable D4 (Doctor & Staff Module): Department hierarchy, doctor profile management, shift schedulling and leave tracking.
- Deliverable D5 (Appointment Engine): Slot generation, booking workflows, conflict locking and real-time appointment status updates.
- Deliverable D6 (Billing & Invoicing): Itemized fee calculation, multi-bracket GST tax handling, discounts and server-side PDF invoice printing.
- Deliverable D7 (Admin Dashboard): Executive KPIs, daily appointment counts, revenue analytics and PDF reconciliation exports.
- Deliverable D8 (Verification & QA): 22 Jest test suites, MongoMemoryServer in-memory testing and Cypress end-to-end flows.
- Deliverable D9 (Project Closure): SPM portfolios (T1-T15), API contracts, deployment guides and user handover manuals.

2.2 Sizing, COCOMO II and Make-vs-Buy Synthesis

Software sizing and estimation synthesized results from T6, T7 and T8:

Estimation Model	Input Parameters / Sizing Basis	Calculated Baseline Value	Actual Project Realization
Function Point Analysis (FPA)	5 External Inputs, 4 External Outputs, 5 Inquiries, 4 Internal Files, 2 External Interfaces. VAF = 0.98.	290.08 Adjusted Function Points (AFP)	Directly guided module interface contracts and sprint story point sizing.
Use Case Points (UCP)	UAW = 8, UUCW = 100, UUCP = 108, TCF = 0.99, EF = 1.343. Total UCP = 143.60.	2,872 Person-Hours (~17.95 PM nominal)	Calibrated for academic team productivity using a factor of 20 person-hours per UCP.
COCOMO II (Semi-Detached)	Size = 28.5 KLOC, a = 3.0, b = 1.12, c = 2.5, d = 0.35. Effort = 3.0 * (28.5)^1.12.	8.00 Person-Months nominal effort; 16.0 weeks schedule duration.	Actual effort: 8.86 Person-Months (utilized 10.7% buffer due to billing tax complexity).
Make-vs-Buy Decision	Evaluated in-house build vs SaaS for payment gateway and	Decision: Hybrid. Build core clinical modules in-house; buy	Saved 1.5 Person-Months of custom payment gateway

Estimation Model	Input Parameters / Sizing Basis	Calculated Baseline Value	Actual Project Realization
	SMS/WhatsApp communications.	Razorpay and Twilio APIs.	compliance and gateway maintainance.

3. Scheduling, Earned Value Tracking and Fast-Track Recovery

The project schedule was established in T5 using the Critical Path Method (CPM) across an Activity on Node (AON) network of 23 activities. The primary critical path was 16.0 weeks long: Inception (2.0w) -> Core Architecture (2.0w) -> Doctor/Patient Schema (2.0w) -> Appointment Engine (2.0w) -> Billing Engine (3.0w) -> Integration & QA (3.0w) -> Project Closure (2.0w).

3.1 Earned Value Tracking at Week 12 Status Date

Project monitoring at Week 12 (T10) revealed an emerging schedule slip and cost variance during the implementation of the complex billing engine:

EVA Metric Name	Abbr.	Formula Applied	Calculated Week 12 Value	RAG Indicator
Budget at Completion	BAC	Total Planned Effort	8.00 Person-Months	Baseline Target
Planned Value	PV	Planned % * BAC (12 / 16 * 8.00)	6.00 Person-Months	Baseline Plan
Earned Value	EV	Verified Completed Work % * BAC	5.60 Person-Months	Amber (Deviation)
Actual Cost	AC	Actual Logged Effort Expended	6.20 Person-Months	Amber (Effort Overrun)
Schedule Variance	SV	EV - PV (5.60 - 6.00)	-0.40 Person-Months	Amber (-0.5 week slip)
Cost Variance	CV	EV - AC (5.60 - 6.20)	-0.60 Person-Months	Amber (Extra debugging)
Schedule Performance Index	SPI	EV / PV (5.60 / 6.00)	0.933 (~0.93)	Amber Alert
Cost Performance Index	CPI	EV / AC (5.60 / 6.20)	0.903 (~0.90)	Amber Alert
Estimate at Completion	EAC	BAC / CPI (8.00 / 0.903)	8.86 Person-Months	Within 10.7% Reserve

3.2 Fast-Track Schedule Recovery Plan

To prevent the -0.40 PM (-0.5 week) schedule slip from delaying final milestone delivery, we executed a fast-track recovery plan during Weeks 13 and 14:

1. Workstream Parallelization: While Urva finalized backend billing unit tests and database concurrency locks, Rakshit commenced frontend admin dashboard screens concurrently rather than waiting for backend sign-off.
2. Scope Freeze Enforcement: All proposed scope additions (such as inpatient bed tracking and ward occupancy maps) were strictly frozen and deferred to Version 2.0.
3. Critical Defect Sprint: Critical defects D-019 (billing item recalculation) and D-024 (pharmacy stock race) were resolved within a 48-hour emergency sprint window.
4. Buffer Utilization: Sprint 7 had been scheduled with 1.0 week of buffer for documentation and testing. Absorbing 0.5 week of buffer realigned our Schedule Performance Index from 0.93 to 1.00 by Week 15.

4. Risk Management and Mitigation Performance

Risk management (T9) identified, assessed and tracked twelve project, technical and operational risks. The table below details the performance of mitigation strategies for our four highest-ranked risks:

Risk ID	Risk Description	Initial Score	Mitigation Strategy Implemented	Residual Score	Outcome Status
R-07	Race condition in concurrent appointment booking causes double-booked doctor time slots.	High (16)	Implemented MongoDB conditional atomic updates and unique compound index on { doctorId, slotTime }.	Low (4)	Resolved. Zero double-booking errors observed under load testing.
R-03	Scope creep from clinical requests for inpatient ward and bed occupancy features delays release.	High (15)	Enforced strict change control board governance; documented inpatient requests in formal v2.0 backlog.	Low (3)	Resolved. Preserved core 4-module delivery within 16-week window.
R-01	TypeScript type mismatch between backend Express endpoints and frontend TanStack React Query hooks.	Medium (12)	Created shared types package and enforced automated npm run build compiler verification in CI pipeline.	Low (2)	Resolved. Zero contract mismatch runtime errors in production.
R-10	Unauthorized clinical access or patient health record leakage across patient accounts.	Critical (20)	Enforced server-side RBAC middleware and injected req.ownOnly database filtering on all patient endpoints.	Low (4)	Resolved. Audited and confirmed in T11 security audit.

5. Quality Assurance, Quality Control and Audit Outcomes

Quality engineering synthesized SQA process controls (T12), automated QC testing and the formal project audit (T11).

5.1 Testing Rigor and Code Coverage

The project achieved a 100% automated test pass rate across 22 test suites containing 340 distinct unit and integration tests. All backend tests execute against MongoMemoryServer in-memory databases with zero external network dependencies, ensuring complete test reproducibility. Teardown hooks explicitly close Redis clients and Socket.io listeners, achieving zero open handle warnings (--detectOpenHandles).

5.2 SQA Audit and Defect Metrics

The formal project audit evaluated 28 checkpoints. Two Major Non-Conformances—legacy JavaScript prototype files and missing doctor leave validation—were resolved via CAPA commits e72547d and ab18eda. The audit compliance index rose from 78.57% to 96.43%.

Key quality metrics achieved at project conclusion are summarized below:

Quality Metric Indicator	Target Standard	Final Achieved Value	Method / Source
Defect Removal Efficiency (DRE)	>= 90.0%	91.43% (32 of 35 defects)	Defect Register calculation (T12)
Statement Code Coverage	>= 85.0%	91.80% (Backend core)	Jest Lcov Coverage report
Branch Code Coverage	>= 80.0%	86.40% (Conditional routes)	Jest Lcov Coverage report
Automated Test Pass Rate	100.0%	100.0% (340 / 340 tests)	Automated CI execution run
Open Asynchronous Handles	0 Leaks	0 Open Handles Detected	Jest --detectOpenHandles
Software Audit Compliance	>= 95.0%	96.43% Post-CAPA	T11 Project Audit Checklist
Defect Density	<= 5.0 / KLOC	4.27 Defects / KLOC	Total defects / 8.2 KLOC written
System Availability Uptime	>= 99.8%	99.85% (Pilot run)	Pilot operational log (720 hrs)

6. Software Configuration Management and Release History

As established in T14, all engineering activities adhered to GitFlow branching conventions. Branch protection rules on main enforced mandatory peer reviews, clean TypeScript compilation and 100% test pass rates prior to merging. Direct commits to main were prohibited. Over the project lifecycle, 84 tracked commits and 18 pull requests were successfully integrated.

The project evolved through seven Semantic Versioning release tags: v0.1.0 (Auth setup) -> v0.2.0 (Patient module) -> v0.4.0 (Doctor management) -> v0.5.0 (Appointment booking) -> v0.6.0 (Billing engine) -> v0.9.0 (Integrated RC-1) -> v1.0.0 (General Availability release candidate). Release v1.0.0 represents the final, fully verified production baseline.

7. Project Closure, Deliverable Verification and Post-Mortem

Project closure formalizes deliverable verification, variance analysis, post-mortem lessons learned and customer handover.

7.1 Deliverable Handover Matrix

WBS Code	Deliverable Name	Verification Artifact	Client / Evaluator Sign-off
D1.0	Requirements Specification	IEEE 830 SRS & TRACEABILITY.md	Approved & Signed
D2.0	System Architecture	Modular TS codebase, ER diagram	Approved & Signed
D3.0	Patient Management Module	Verified API routes & Patients.tsx UI	Approved & Signed
D4.0	Doctor & Staff Module	Staff controller & Staff.tsx UI	Approved & Signed
D5.0	Appointment Scheduling Engine	Slot booking, conflict lock & slip PDF	Approved & Signed
D6.0	Billing & Invoicing Engine	Itemized invoice & PDF generator	Approved & Signed
D7.0	Admin Dashboard & Reports	Analytics routes & revenue export	Approved & Signed
D8.0	Quality & Test Suites	22 test suites (340 tests pass)	Approved & Signed

WBS Code	Deliverable Name	Verification Artifact	Client / Evaluator Sign-off
D9.0	SPM Portfolios & Closeout	Tutorials T1-T15 & Handover Manual	Approved & Signed

7.2 Project Variance Analysis

Comparing baseline plans against final realization:

Project Parameter	Planned Baseline	Actual Realization	Variance Delta	Operational Explanation
Project Duration	16.0 Weeks	16.0 Weeks	0.0 Weeks (0%)	Recovered 0.5 week slip using fast-track parallelization in Sprint 6-7.
Engineering Effort	8.00 Person-Months	8.86 Person-Months	+0.86 PM (+10.7%)	Extra effort in billing tax logic and concurrency locking; fit in reserve.
Financial Cost	₹1,80,000	₹1,99,350	+₹19,350 (+10.7%)	Directly proportional to effort; remained within approved 15% budget cap.
Codebase Size	25,000 LOC	28,500 LOC	+3,500 LOC (+14%)	Added server-side PDF generator and database mock seed scripts.
Test Coverage	>= 85.0%	90.20%	+5.20%	Expanded unit tests for edge-case billing discounts and phone regex.

7.3 Post-Mortem Analysis (Lessons Learned)

A structured post-mortem evaluation identified key engineering successes and areas for organizational improvement:

- What Went Well: Migrating from prototype JavaScript to strict TypeScript with Zod runtime validation prevented countless runtime bugs. In-memory MongoMemoryServer testing enabled fast CI test runs (340 tests in under 12 seconds). SCM branch protection prevented merge overwrites.
- What Could Be Improved: Initial estimation under-estimated the complexity of financial rounding rules across multi-tier GST tax brackets. In addition, date-specific doctor leave logic was omitted from initial user stories, necessitating mid-project CAPA refactoring.
- Future Roadmap: PulseCare HMS v2.0 roadmap includes inpatient bed management, electronic prescription QR codes and automated ABDM Health Locker synchronization.

8. Conclusion

The PulseCare Hospital Management System project successfully demonstrated the end-to-end discipline of Software Project Management. By coupling Agile Scrum development with milestone governance, empirical sizing models (COCOMO II, FPA, UCP), proactive Earned Value tracking, risk mitigation and ISO 9001 audit standards, our two-student team delivered a production-grade clinical platform on schedule at Week 16.0. The

system fulfills all specified requirements, passes 340 automated tests with 90.2% coverage and stands fully documented, audited and ready for clinical deployment.